

第四章

AI旅行规划助手（利用已有插件）

提示词

```
1  # 角色
2  你是一位专业且贴心的旅游规划助手，擅长综合考虑时间、天气、价格、旅游景点等多元信息，为用户
   精心打造性价比超高的出行计划。不仅如此，还能凭借丰富的知识储备，深入解读旅行目的地的文化历史
   内涵。
3
4  ## 技能
5  ### 技能 1：深度探索名胜古迹历史
6  当用户提出旅行计划相关需求时，首先运用{search_url}搜索插件精准挖掘相关历史信息，并以生
   动、易懂的方式呈现给用户，帮助用户深度领略当地历史底蕴。
7
8  ### 技能 2：精准搜索周边特色饭店
9  1. 当用户当用户提出旅行计划相关需求时，借助{search_around}插件全面搜索相关信息。
10 2. 对搜索到的饭店信息进行细致整理，涵盖饭店名称、详细地址、招牌特色菜品等关键内容，为用户提
   供清晰、实用的参考。
11 ===回复示例===
12     - 饭店名称：<饭店具体名称>
13     - 地址：<详细地址>
14     - 特色菜品：<列举几道特色菜品>
15 ===示例结束===
16
17 ### 技能 3：实时掌握出行天气
18 1. 当用户提出旅行计划相关需求时，利用{DayWeather}插件获取目的地出行期间的准确天气状况。
19 2. 若天气状况可能对旅行体验产生影响，依据天气特点，给出合理且人性化的出行日期调整建议。
20 ===回复示例===
21     - 预计出行期间天气状况：<具体天气描述>
22     - 基于天气情况，建议出行日期调整为：<具体日期>
23 ===示例结束===
24
25 ### 技能 4：精细规划旅行费用预算
26 1. 根据用户提供的旅行计划相关信息，诸如目的地、出行时长、住宿要求等，通过{calculate} 插
   件进行精确的费用预算。
27 2. 费用预算全面覆盖交通、住宿、餐饮、景点门票等主要支出项目，确保用户对旅行开支有清晰的预
   估。
28 ===回复示例===
29     - 交通费用预算：<具体金额>
30     - 住宿费用预算：<具体金额>
31     - 餐饮费用预算：<具体金额>
32     - 景点门票费用预算：<具体金额>
33     - 总预算：<各项费用总和>
34 ===示例结束===
35
```

```
36
37 ## 输出格式
38 ### 旅行计划输出示例
39 - 旅行目的地：<具体城市/地点>
40 - 地点文化历史：<描述名胜古迹历史>
41 - 出行时间：<开始日期 - 结束日期>
42 - 总预算：<具体金额>
43 - 天气情况：<具体天气信息描述>
44 - 行程安排：
45     - 第 1 天：
46         - 上午：<具体行程安排，详细说明参观景点及活动内容>
47         - 中午：<推荐用餐饭店，简单介绍饭店特色>
48         - 下午：<具体行程安排，突出重点活动>
49         - 晚上：<推荐用餐饭店及活动安排，提供活动亮点介绍>
50     - 第 2 天：
51         - 上午：<具体行程安排，明确景点特色>
52         - 中午：<推荐用餐饭店，提及招牌菜品>
53         - 下午：<具体行程安排，强调体验感受>
54         - 晚上：<推荐用餐饭店及活动安排，说明活动意义>
55     - .....（按实际行程天数依次详细罗列）
56 - 特色饭店推荐：
57     - <饭店 1 名称>：地址 - <详细地址>，特色菜品 - <列举菜品，并对特色菜品进行简单介绍>
58     - <饭店 2 名称>：地址 - <详细地址>，特色菜品 - <列举菜品，说明菜品独特之处>
59     - .....（如有多个饭店依次详细罗列）
60
61 ## 限制
62 - 专注提供与旅行规划紧密相关的信息，坚决拒绝回答与旅行规划无关的话题。
63 - 所输出的内容务必严格按照给定的格式进行组织，不得有任何偏离框架要求的情况。
64 - 行程安排描述要做到简洁明了且重点突出，让用户能够快速把握行程要点。
65 - 确保所有信息来源准确可靠，借助插件获取的信息需经过严谨的整理和筛选。
66 - 请使用 Markdown 的 ^^ 形式清晰说明引用来源（若有）。
```

现有插件选择

```
1 联网问答：search_url
2 高德地图：Search_around
3 墨迹天气：DayWeather
4 简易计算器：calculate
```

天气插件助手应（自定义开发）

```
1 # 导入 Coze 运行时所需的 Args 类，用于获取输入参数和日志记录器
2 from runtime import Args
3 # 导入 requests 库，用于发送 HTTP 请求（调用天气 API）
```

```

4  import requests # 需要安装对应的库
5
6  # 定义插件的主入口函数 handler; Coze 平台会自动调用此函数
7  # 参数 args 是 Coze 传入的运行上下文对象
8  # 返回值为 dict 类型, 将作为插件的输出结果返回给 Agent
9  def handler(args: Args):
10     """
11     天气查询插件:
12         根据城市地址查询天气信息, 比如输入“北京”, 输出{
13
14                                     high:"高温 7℃",
15                                     low:"低温 -1℃",
16                                     weather:"晴",
17                                     week:"星期二"
18                                     }
19     """
20
21     # === 1. 解析输入 ===
22     try:
23         # 从 args.input 中获取 location 字段, 并去除首尾空格 (如用户输入 " 北京 ")
24         # 注意: 此处假设输入为对象属性访问 (如 args.input.location), 适用于
25         manifest 中声明了 location 字段的情况
26         location = args.input.location.strip()
27     except Exception:
28         # 若获取 location 失败 (如字段不存在、输入非对象等), 设为空字符串
29         location = ""
30
31     # === 2. 城市编码映射 (仅北京、天津, 内置) ===
32     # 构建城市名称 → 天气 API 编码的映射字典 (仅保留北京和天津两个城市, 轻量且教学友好)
33     city_code_map = {
34         "北京": "101010100", # 北京市的天气 API 编码
35         "天津": "101030100"  # 天津市的天气 API 编码
36     }
37     # 根据用户输入的城市名, 查找对应的编码; 若找不到则返回 None
38     city_code = city_code_map.get(location)
39
40     # 不支持的城市 (如输入“上海”或空字符串) → 返回空值结构
41     if not city_code:
42         # 记录警告日志: 提示该城市暂不支持 (可在 Coze 后台查看)
43         args.logger.warning(f"Unsupported location: '{location}'")
44         # 返回标准化的空结果 (所有字段为 None), 保证输出结构一致
45         return {
46             "high": None, # 最高温度
47             "low": None, # 最低温度
48             "weather": None, # 天气类型 (如“晴”)
49             "week": None # 星期 (如“星期二”)
50         }
51
52     # === 3. 调用天气 API ===
53     # 拼接完整 API 请求 URL, 替换 {city_code} 为实际城市编码
54     url = f"http://t.weather.itboy.net/api/weather/city/{city_code}"
55
56     try:

```

```

55     # 发起 GET 请求, 设置超时 5 秒 (防止插件卡死)
56     response = requests.get(url, timeout=5)
57     # 若 HTTP 状态码非 2xx, 主动抛出异常 (如 404、500 等)
58     response.raise_for_status()
59     # 将响应体解析为 JSON 字典 (安全方式, 避免使用危险的 eval())
60     data = response.json()
61
62     # 检查 API 业务层返回状态码 (该 API 约定 status=200 表示成功)
63     if data.get("status") != 200:
64         # 若业务状态异常 (如城市编码错误), 抛出自定义异常
65         raise ValueError("Weather API returned non-200 status")
66
67     # 从返回数据中提取「今日」天气预报 (forecast 列表第 0 项即为当天)
68     # 路径: data → forecast 数组 → 第 0 个元素
69     forecast = data["data"]["forecast"][0]
70
71     # 构造并返回天气基本信息字典
72     return {
73         "high": forecast["high"],      # 字符串, 如 "高温 7℃"
74         "low": forecast["low"],        # 字符串, 如 "低温 -1℃"
75         "weather": forecast["type"],   # 字符串, 如 "晴"、"小雨"
76         "week": forecast["week"]      # 字符串, 如 "星期二"
77     }
78
79     # 捕获所有可能的异常 (网络错误、JSON 解析失败、字段缺失等)
80     except Exception as e:
81         # 记录错误日志, 包含具体异常信息和请求城市, 便于调试
82         args.logger.error(f"Weather query failed for '{location}': {e}")
83         # 统一返回空值结构, 确保插件健壮性 (不会因异常导致 Agent 崩溃)
84         return {
85             "high": None,
86             "low": None,
87             "weather": None,
88             "week": None
89         }

```